Software Engineering for AI/AS: AI Engineering and LLM-Guided Malware Analysis

[Nicolò] [Rosa]
WASP Software Engineering Course, 2026 Module

1 Introduction

Malware, malicious software designed to compromise computers and devices, has grown enormously in both scale and diversity, particularly on Linux, which today runs everything from cloud servers to home routers and other embedded devices. To understand what a piece of malware actually does, security analysts commonly rely on dynamic analysis: running the sample inside an isolated, disposable environment, or "sandbox", and observing its behavior, such as which files it touches, which network connections it opens, and which privileges it tries to gain, instead of only reading its code.

The problem is that a sandbox is never a neutral observer. Malware frequently checks properties of the environment it runs in before deciding whether, and how, to act: it may look for signs of virtualization, probe for specific configuration files, wait for a particular kernel version, or expect a real command-and-control server to answer. If any such check fails, the sample can suppress its malicious behavior entirely, leaving the analyst with an incomplete and misleading picture. Because Linux malware spans an unusually wide range of families and behaviors, from IoT botnets to cryptominers, there is no single environment configuration that works for every sample. Building the "right" sandbox for a given sample still largely depends on an analyst manually inspecting its behavior, forming a hypothesis about what might be missing, adjusting the environment by hand, and re-running the sample: a slow, expertise-heavy loop that does not scale to the volume of malware seen today. Existing automated tools either apply the same generic countermeasures to every sample or search the space of possible environments blindly, without reasoning about what a specific sample's own behavior suggests it actually needs.

My research investigates whether this manual reasoning step can instead be delegated, at least partially, to large language models (LLMs). The core idea is to give an LLM the same kind of evidence an analyst would look at, a summarized trace of the system calls the malware attempted, and let it propose targeted, sample-specific changes to the sandbox environment, which are then applied before re-executing the sample. The new behavior is compared against the previous run, and the loop repeats: the model effectively takes the place of the analyst's iterative "observe, hypothesize, adjust" cycle, but automatically and at scale. I implement and evaluate this idea in a framework, LEA [1], built around a custom kernel module for tracing and environment manipulation, and show on a corpus of real-world Linux malware that this sample-specific, LLM-guided adaptation reveals behavior that neither an unmodified sandbox nor a fixed set of generic anti-evasion rules exposes.

2 Lecture Principles

2.1 Principle 1: AI Testing Challenges

Lecture 2 lists four challenges that make testing AI-based systems harder than testing traditional software: non-determinism (the same input can give different outputs), ambiguous oracles (there is no precise specification of the "correct" output to compare against), opacity (the system behaves as a black box), and complexity (many interacting components and configuration choices, so behavior is sensitive to small changes). All four apply directly to LEA. Malware execution itself is non-deterministic: timing, process scheduling, and address-space randomization mean the same sample can behave slightly differently across two runs of an unmodified sandbox, so before trusting any change we first measure

this run-to-run noise across repeated executions. There is also no oracle for the "correct" behavior of a malware sample: nobody has a ground-truth specification of what it should do, so instead of comparing against a known-correct answer, LEA compares behavior across configurations (unmodified sandbox vs. fixed rules vs. LLM-guided) and treats newly observed activity as a signal rather than a verified fact. The LLM controller is itself opaque, which is why LEA's evaluation includes an explicit grounding check that inspects, after a rule has already been applied, whether it was actually supported by evidence already in the trace, rather than just trusting the model's reasoning at face value. Finally, complexity and configuration-sensitivity are exactly what the ablation study in the paper measures: removing one category of environment modification at a time shows which parts of the configuration space are actually responsible for the exposed behavior.

2.2 Principle 2: Preserve vs. Purify

The keynote "Agentic Software Engineering Will Eat the World" argues that as agents take over more implementation work, some engineering practices should be preserved even as others are purified away; among the practices to preserve are explicit reasoning about constraints, validation and verification, and traceability between intent, implementation, and behavior. This framing describes LEA's own design choices well. Instead of giving the LLM free control over the sandbox, LEA restricts it to a fixed set of eight modification categories decided in advance: an explicit constraint the model must reason within, rather than an open-ended capability. The grounding check from Principle 1 is itself a direct instance of "validation and verification" applied to an agent's output, not just to hand-written code. And each iteration explicitly carries forward which modification was proposed, why, and what trace resulted, preserving traceability between the model's intent and the sample's observed behavior across attempts, instead of treating every LLM decision as a disposable, unaccountable one.

3 Guest-Lecture Principles

3.1 Principle 1: AI-as-Component vs. AI-as-Developer

Guest lecturer Julian Frattini described requirements engineering, and highlighted a shift he called AI-as-component versus AI-as-developer. Traditionally, AI is a component that a system is built around: engineers write requirements for it, the same way they would for any other part of the system. Increasingly, though, AI is not just a component: it is doing part of the requirements-engineering work itself, deciding "how" to solve a problem, while humans focus on the "why" and the "what". LEA is an example of this second, less common case. The LLM controller is not a classifier plugged into a fixed pipeline; it makes the same decisions an analyst would normally make, choosing how to adapt the sandbox at each iteration. What stays with the human designers is the why and the what: the threat model fixes what LEA is allowed to try to expose, and the fixed set of eight modification categories fixes what kinds of changes the model is allowed to propose. The LLM decides the specific "how" within those boundaries, but it does not get to set the boundaries themselves.

3.2 Principle 2: Hidden Gaps Become Visible

Guest lecturer Per Lenberg argued that as AI takes over tasks that used to rely on human interpretation and informal coordination, gaps that used to stay hidden become visible: unclear decisions, unclear responsibilities, unclear reasons for stopping. This matches something LEA's own evaluation shows. When a human analyst manually tries to unlock a stubborn malware sample and eventually gives up, the real reason usually stays informal and undocumented: maybe they ran out of time, maybe they guessed wrong, maybe the sample genuinely cannot be unlocked locally at all. Because LEA repeats the same explicit process automatically, this distinction becomes visible instead of staying a guess. In one case study, LEA's iterations progressively remove local blockers, one after another, until execution stalls specifically because the malware needs to talk to a real, external command-and-control server, something no local environment change can fix. The boundary between "solvable by adapting the sandbox" and "not solvable locally" stops being an informal analyst intuition and becomes an explicit, documented, and reproducible finding instead.

4 Data Scientists, Software Engineers, and AI Engineers

What differences do the chapters describe between data scientists and software engineers, and do I agree with this? The book [3] describes data scientists as trained in statistics and machine learning, working in an exploratory, notebook-based style, and judging their work mostly by accuracy on held-out data. Software engineers, by contrast, focus on delivering a product that meets user needs within a budget: gathering requirements, designing an architecture, implementing and testing it, deploying it, and maintaining it over time, while constantly trading off qualities like cost, security, and maintainability. I agree with this distinction. In my experience these are two genuinely separate roles, with different training, different daily work, and different definitions of what counts as "done".

How does the system-wide view described in the chapters apply to my own research? LEA is a good illustration of this point. The large language model itself, DeepSeek V4 or MiniMax M3, is a swappable, general-purpose component; it is not where the actual engineering contribution lives. What makes LEA work is everything around the model: the kernel module that safely collects and fabricates evidence, the logic that decides what trace evidence the model is even shown under a limited token budget, the loop that re-executes the sample in a fresh sandbox after every change, and the grounding check used afterward to measure how often the model's outputs are actually supported by evidence. If I swapped in a different LLM, the paper's main findings would likely not change much; if the kernel module broke, the whole system would.

Will the roles stay distinct, and where does "AI Engineering" fit? I think data scientists and software engineers will remain distinct as base disciplines and educations, with only a thin overlap between them: the underlying skills and mindsets are simply different. "AI Engineer" is not a merger of the two, but a bridging role that people from either side can grow into: it needs the software-engineering discipline to build and operate the surrounding pipeline, and enough data-science judgment to tell whether the data going into the model, and coming out of it, is actually sound. LEA reflects this split rather than a merge: choosing between DeepSeek V4 and MiniMax M3, and judging whether a proposed rule is grounded in evidence, is data-science-flavored work; the kernel module, the tracer, and the evaluation harness are software-engineering work. The same project, two different kinds of work.

How do foundation models and agentic workflows change these boundaries? In LEA, prompting is not the lightweight, exploratory task the book associates with data science; it is systems work. Deciding what trace evidence to prioritize under a fixed token budget, how to summarize the history of previous attempts, and how to validate an output before acting on it are all requirements-engineering and quality-assurance problems wearing a prompt-shaped disguise. Agentic workflows push this further: once a model's output triggers an action in the next iteration, as it does in LEA, prompt design stops being a data-science detail and becomes part of the system's architecture, with its own correctness and reliability concerns.

5 Paper Analysis

5.1 SETA: Statistical Fault Attribution for Compound AI Systems (CAIN 2026)

Core ideas and their SE importance SETA [2] studies what it calls compound AI systems: pipelines built from several connected components rather than one single model. Existing robustness testing tools work well for a single model, but they cannot tell you which specific component in a pipeline is responsible when the whole system misbehaves. SETA fixes this by testing each component on its own. It defines a small property for each component, called a metamorphic relation, something that should stay true even after a small, controlled change to the input. If a component's property breaks while the overall system fails, that component gets some of the blame. SETA aggregates this across many test runs into a Failure Contribution score per component, so an engineer learns which part of the pipeline is actually unreliable instead of only knowing that the pipeline failed somewhere. This is a concrete answer to the

oracle problem raised earlier in this essay: how do you test a system when there is no ground truth for what a correct output looks like.

Relation to my research LEA is also a pipeline with several connected parts: the kernel module tracer, the LLM that proposes environment changes, the grounding check, and the loop that re-executes the sample. My own ablation study already asks a SETA-shaped question. Removing one modification category and checking whether a previously observed behavioral gain disappears is a manual, one-off version of attributing a system level result to a specific component. SETA replaces this ad hoc process with a repeatable, statistical one, run across many samples instead of one comparison at a time.

Integration into a larger AI-intensive project Imagine a SOC triage platform that receives incoming malware samples and passes them through several AI-driven stages, an environment-adaptation stage like LEA, a stage that labels the exposed behavior, and a stage that writes a short report for the analyst. If an analyst flags a wrong verdict, the platform needs to know which stage caused it. SETA-style attribution would give each stage its own metamorphic relation, for example checking that a small, harmless change to the trace should not flip the environment-adaptation stage's proposed rules, and use the resulting Failure Contribution scores to point the analyst to the actual faulty stage instead of the whole pipeline.

Adaptation of my research I would replace LEA's current one-category-at-a-time ablation with a proper Failure Contribution score, computed across the full sample set rather than the 40 samples used in LEA's ablation study. This means defining a metamorphic relation for each of LEA's own components, which is harder than it sounds, since LEA's components are not identical neural modules like SETA's vision pipeline. They are a tracer, an LLM call, and a rule application step, so the property that should stay invariant looks different for each one and would need to be designed from scratch.

5.2 LUMEN: A Systems Approach to LLM-Guided Activation of Hidden Behaviors in Malware (DSN 2026)

Core ideas and their SE importance LUMEN [4] targets Windows malware and faces the same underlying problem as LEA: malware hides its worst behavior from sandboxes, and analysts have to manually work out why. Its approach is different in kind. Instead of changing what the sandbox reports, LUMEN finds the exact branch instruction in the malware's code that leads to the hidden behavior, then either prepares the environment, supplies command line arguments, or directly forces execution down the other path using a debugger. A key design choice is that LUMEN's LLM does not run an open-ended loop. It follows a fixed two-stage workflow, first locate the decision point, then choose how to activate it, because the authors found that LLMs perform worse on long, open-ended planning tasks. This is a direct, evaluated example of a broader lesson: giving an LLM more autonomy is not automatically the right choice, even outside malware analysis.

Relation to my research LEA and LUMEN attack the same problem with opposite philosophies. LEA only changes what the environment reports and lets the sample run its own course, repeating this over up to three attempts that build on each other. LUMEN instead directly forces a specific branch when needed, but only ever in one shot. LUMEN's own results note that samples needing several, independent decision point changes are hard for a single-shot method, which is exactly the case LEA's iteration is built for. The two papers also hit the same wall independently. Samples that depend on a missing file or an unreachable remote server fall outside what either system can fix: LUMEN's Remexi and Electricfish samples, and LEA's own case study on the boundary of local adaptation described earlier in this essay, are the same limitation found twice.

Integration into a larger AI-intensive project In the SOC platform, LUMEN's forced execution gives a genuine escalation path once environment adaptation stalls. If the LEA-style stage makes progress but gets stuck on a stubborn decision point, the platform could fall back to a LUMEN-style step: locate

the exact branch and force the alternate path, rather than continuing to guess at environment changes. LUMEN's own evaluation shows this fallback is genuinely useful, since one of their three models leaned on forced execution far more than the other two; the paper also cautions that forcing execution past a blocker only guarantees short-term progress, and can still fail later if the real underlying cause, such as a missing dependency, was never addressed.

Adaptation of my research LUMEN's insistence on a narrow, non-agentic role for the LLM is worth taking seriously, since LEA is explicitly iterative and lets the model decide what to try next. Rather than giving that up, LEA could borrow LUMEN's earlier verification step. LUMEN checks an LLM's answer for internal consistency before it is ever used, while LEA's grounding check only runs after a rule has already been applied. Moving some of that checking earlier could reduce the share of ungrounded rules already discussed earlier in this essay. LEA could also borrow LUMEN's habit of explicitly giving up and reporting a blocker as out of reach, such as a missing file or an unreachable server, instead of quietly spending another iteration on it.

6 Evidence, Originality, and GenAI-Use Transparency

My own LEA paper [1] is currently under anonymous submission and has not completed peer review, though I am its author and know its methodology first-hand. Of the two papers in Section 5, LUMEN [4] is a peer-reviewed paper published at DSN 2026; SETA [2] is a peer-reviewed paper accepted at CAIN 2026, which I read through its arXiv preprint copy since the proceedings version was not yet available. I trust the peer-reviewed conference papers the most, since they went through independent review, and treat the slides and my own not-yet-reviewed paper as reliable but secondary sources.

I used Claude throughout the drafting of this essay: to read and summarize the assigned slides and book chapters, to search for and shortlist candidate papers for Section 5, and to produce prose drafts for each section based on the principles, opinions, and choices I gave it. Which lecture and guest-lecture principles to use, the answers to the personal-opinion questions in Section 4, the choice of SETA and LUMEN, and the SOC-platform framing in Section 5 are my own decisions. I reviewed, edited, and take responsibility for every claim, citation, and piece of wording in this essay.

References

[1] Anonymous. LEA: LLM-guided environment adaptation to expose hidden malware behavior. In submission to ASIACCS 2027, 2027.

[2] Sayak Chowdhury and Meenakshi D'Souza. SETA: Statistical fault attribution for compound AI systems. In *2026 IEEE/ACM 5th International Conference on AI Engineering – Software Engineering for AI (CAIN)*, pages 1–11, Rio de Janeiro, Brazil, 2026.

[3] Christian Kästner. *Machine Learning in Production: From Models to Products*. MIT Press, 2025. Chapters 1–2, https://mlip-cmu.github.io/book/.

[4] Kevin Valakuzhy, Miuyin Yong Wong, Douglas M. Blough, Mustaque Ahamad, and Fabian Monrose. LUMEN: A systems approach to LLM-guided activation of hidden behaviors in malware. In *2026 56th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2026.